

Practical Work 5: The Longest Path (MapReduce)

Vo Truong Giang - BI12-130

December 11, 2025

1 Introduction

In this practical work, I utilized the MapReduce paradigm to solve the "Longest Path" problem. The goal is to find the file path with the maximum length from a large dataset of file paths.

2 System Design

I implemented a simplified MapReduce framework in C.

- **Input:** A large file generated by `find /`, split into smaller chunks to simulate distributed nodes.
- **Mapper:** Reads a chunk of file paths and identifies the single longest path within that chunk.
- **Reducer:** Collects the local maximums from all Mappers and determines the global maximum.

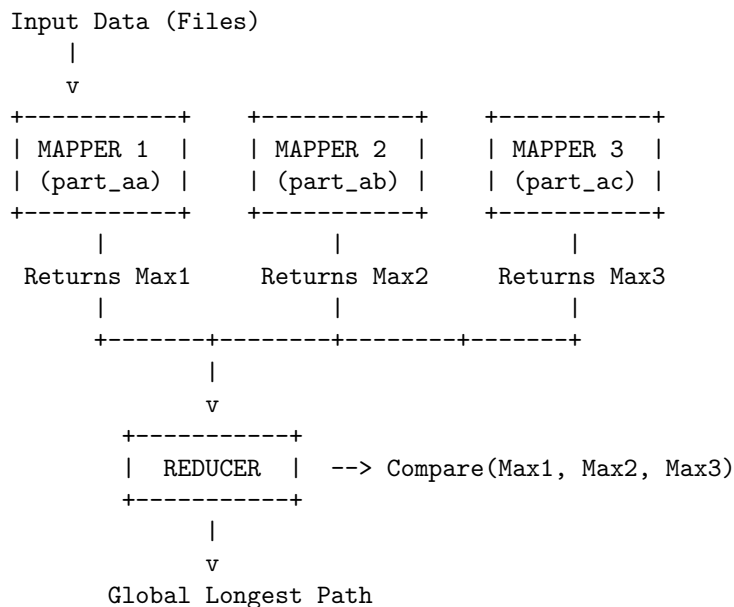


Figure 1: MapReduce Workflow for Longest Path

3 Implementation Logic

3.1 Mapper

The mapper iterates through lines in a file, keeping track of the longest string seen so far.

```
if (strlen(line) > max_len) {
    max_len = strlen(line);
    strcpy(longest_local, line);
}
```

3.2 Reducer

The reducer takes an array of strings (outputs from mappers) and finds the maximum length among them.

4 Test and Validation

To verify the correctness of the system, I conducted a test using real system files from the WSL environment.

4.1 Test Setup

First, I generated the dataset and split it into 3 parts to simulate a distributed environment with 3 worker nodes:

1. **Generate Data:** Scanned the `/usr` directory.

```
find /usr -type f > all_paths.txt
```

2. **Simulate Distribution:** Split the data into 3 chunks.

```
split -n 1/3 all_paths.txt part_
```

This created 3 files: `part_aa`, `part_ab`, and `part_ac`.

4.2 Execution and Result

I executed the program passing the 3 chunks as arguments. Below is the actual output from the terminal:

```
$ ./longest_path part_*
```

```
--- MAP PHASE ---
```

```
[MAPPER] File 'part_aa' -> Max len: 124
```

```
[MAPPER] File 'part_ab' -> Max len: 130
```

```
[MAPPER] File 'part_ac' -> Max len: 113
```

```
--- REDUCE PHASE ---
```

```
[REDUCER] Processing 3 candidates -> Global Max len: 130
```

```
=== FINAL RESULT ===
```

```
Longest Path: /usr/lib/python3/dist-packages/oauthlib/oauth2/rfc6749/  
grant_types/__pycache__/resource_owner_password_credentials.cpython-312.pyc  
Length: 130
```

(Note: The longest path above has been split into two lines for readability in this report).

4.3 Analysis

The result shows that the Mapper processes correctly identified the local maximum of each chunk (124, 130, 113). The Reducer then correctly determined that 130 is the global maximum and returned the specific Python cache file path.

5 Conclusion

The MapReduce approach efficiently reduces the problem scale. Instead of comparing every path globally, we process chunks in parallel (simulated) and only aggregate the "champions" of each chunk.